

LAB 2: HASHING

INFOE 3

Members:

Joy Melisa S. Segotier

Jorlyn Shane M. Claveria

Maria Theresa N. Reyes

Post Questions:

1. In this specific activity, which one is the Subject (the entity that acts to conduct the check) and which is the Object (the entity that is the target of the check)?

Answer:

The Python Script (the Integrity Auditor) is the **Subject** which is the one performing the "audit" by reading the file and calculating the hash, while the database.csv file (the Asset) is the **Object** which is the target of the integrity check.

2. If you manually open the file and change one variable (from Instructions), is that action considered a Vulnerability or a Threat Event?

Answer:

If you manually open the file and change one variable that action is considered a Threat Event because by manually opening the file and changing the data, you are simulating a "Threat Agent" performing an action that causes a loss of integrity.

Screenshots:

This screenshot shows the Visual Studio Code interface with the 'database.csv' file open. The file contains the following data:

1	Name,AccountNumber,Balance
2	Juan Dela Cruz,1001,500
3	Maria Santos,1002,100
4	Pedro Reyes,1003,750

The terminal at the bottom shows a PowerShell prompt: `PS C:\Users\HYUN GADGETS\Desktop\Infoe_3_lab2_hashing>`. The status bar at the bottom indicates the current cursor position: `Col 1: Name Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} CSV`.

Here is the original unmodified database.csv

This screenshot shows the Visual Studio Code interface with the 'database.csv' file open. The file contains the following data:

1	Name,AccountNumber,Balance
2	Juan Dela Cruz,1001,500
3	Maria Santos,1002,100
4	Pedro Reyes,1003,7500

The terminal at the bottom shows a PowerShell prompt: `PS C:\Users\HYUN GADGETS\Desktop\Infoe_3_lab2_hashing>`. The status bar at the bottom indicates the current cursor position: `Col 3: Balance Ln 4, Col 22 Spaces: 4 UTF-8 CRLF {} CSV`.

Here is the manually modified database.csv

```
format.ipynb • database.csv
format.ipynb > # 1st cell - Hash
Generate + Code + Markdown Run All Clear All Outputs Outline Python 3.14.2
with open(path, 'rb') as f:
    for byte_block in iter(lambda: f.read(4096), b''):
        sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()

# Generate and print the result
try:
    print(f"File: {filename}")
    print(f"SHA-256 Hash: {get_file_hash(filename)}") # Copy the hash and paste it
    print("-" * 30)
except FileNotFoundError:
    print(f"Error: '{filename}' not found.")

[1] ✓ 0.0s Python
File: database.csv
SHA-256 Hash: 78a6ebd4c86d44738b31df5cc558e5503684c9ccb698aab4d8e1ba7ec0801066
-----

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... powershell + - - - - -
t be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to
re-enable it, run 'Import-Module PSReadLine'.

PS C:\Users\HYUN GADGETS\Desktop\Infoe_3_lab2_hashing>
```

This screenshot shows the execution of the first cell, where the original SHA-256 hash of the database.csv file is generated. This "digital fingerprint" represents the baseline for the file's integrity before any changes are made.

```
format.ipynb • database.csv
format.ipynb > # 2nd cell
Generate + Code + Markdown Restart Clear All Outputs Jupyter Variables Python 3.14.2
if not os.path.exists(filename):
    print(f"ALERT: Asset '{filename}' is missing! Integrity cannot be verified.")
    return

# B. THE FINGERPRINTING
sha256_hash = hashlib.sha256()
try:
    with open(filename, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
        current_hash = sha256_hash.hexdigest()
except Exception as e:
    print(f"Error reading file: {e}")
    return

# C. THE TECHNICAL CONTROL
if current_hash == EXPECTED_HASH:
    print("Integrity Verified: The file is safe and has not been modified.")
else:
    print("!!! INTEGRITY BREACH DETECTED !!!")
    print(f"Expected: {EXPECTED_HASH}")
    print(f"Current: {current_hash}")
    print("The file has been tampered with by a Threat Agent.")

# Run the function
run_integrity_audit()

[7] ✓ 0.0s Python
Integrity Verified: The file is safe and has not been modified.
```

The output of the auditor script when the file has not been changed. The current hash matches the EXPECTED_HASH, confirming that the object's integrity is intact.

```
format.ipynb • database.csv
format.ipynb > M4 2 Ways to simulate
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Python 3.14.2

# B. THE FINGERPRINTING
sha256_hash = hashlib.sha256()
try:
    with open(filename, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    current_hash = sha256_hash.hexdigest()
except Exception as e:
    print(f"Error reading file: {e}")
    return

# C. THE TECHNICAL CONTROL
if current_hash == EXPECTED_HASH:
    print("Integrity Verified: The file is safe and has not been modified.")
else:
    print("!!! INTEGRITY BREACH DETECTED !!!")
    print(f"Expected: {EXPECTED_HASH}")
    print(f"Current: {current_hash}")
    print("The file has been tampered with by a Threat Agent.")

# Run the function
run_integrity_audit()
```

[6] ✓ 0.0s Python

!!! INTEGRITY BREACH DETECTED !!!
Expected: 78a6ebd4c86d44738b31df5cc558e5503684c9ccb698aab4d8e1ba7ec0801066
Current: 248c7bcd62d5782a3aa949f769e71fd5b855e9f4a295dcdb9d362843f7c42a2
The file has been tampered with by a Threat Agent.

This screenshot captures the auditor triggering a high-priority alert. Because the file was modified, the calculated hash no longer matches the protection profile, successfully detecting the loss of integrity.